

- Description

This module manages an elevator load and energy optimization system, ensuring that:

1. **The total weight of passengers is monitored to prevent overload situations.**
2. **The system calculates the estimated energy consumption based on the current load.**
3. **If the energy usage exceeds a threshold, an optimization suggestion is provided.**
4. **A log of all load measurements, energy usage, and efficiency recommendations is maintained for analysis.**

This system helps improve elevator efficiency, reduce energy waste, and ensure safe load management.

- Specification

module ElevatorLoadAndEnergyOptimization

var currentLoad, maxLoad, energyUsage, energyThreshold, travelDistance: real

var log: sequence of string

process enterElevator(weight: real)

functional scenario form:

currentLoad ≤ maxLoad && log = log ⊕ ("Load Updated: " + currentLoad + " kg"), Passenger entered, load updated.

||

currentLoad > maxLoad && log = log ⊕ ("Overload Warning: " + currentLoad + " kg, Reduce Load Immediately."), Overload warning triggered, reduce load.

end process

process calculateEnergyUsage(travelDistance: real)

functional scenario form:

(energyUsage ≤ energyThreshold) && log = log ⊕ ("Energy Consumption Recorded: " + energyUsage + " kWh"), Energy usage recorded successfully.

OR

(energyUsage > energyThreshold) && log = log ⊕ ("High Energy Usage Alert: " + energyUsage + " kWh, Consider Load Optimization.") High energy usage alert triggered, consider optimization.

end process

end module

- Java Code

```
package code;
import java.util.*;
class ElevatorSystem {
    private double maxLoad;
    private double energyThreshold;
    private double currentLoad;
```

```

private List<String> log;
public ElevatorSystem(double maxLoad, double energyThreshold) {
    this.maxLoad = maxLoad;
    this.energyThreshold = energyThreshold;
    this.currentLoad = 0.0;
    this.log = new ArrayList<>();
}
public void enterElevator(double weight) {
    currentLoad += weight;
    String message = String.format("Current Load: %.1f kg", current
Load);
    if (currentLoad > maxLoad) {
        message += " | WARNING: Overload Detected! Reduce Load Imme
diately.";
    }
    System.out.println(message);
    log.add(message);
}
public void calculateEnergyUsage(double travelDistance) {
    double energyUsage = (currentLoad / maxLoad) * travelDistance *
2;
    String message = String.format("Estimated Energy Usage: %.2f kW
h for %.1f meters travel", energyUsage, travelDistance);

    if (energyUsage > energyThreshold) {
        message += " | ALERT: High Energy Usage! Consider Load Opti
mization.";
    }
    System.out.println(message);
    log.add(message);
}
public void printLog() {
    System.out.println("\nElevator System Log:");
    for (String logEntry : log) {
        System.out.println(logEntry);
    }
}
}
public class c35_ElevatorOptimization {
    public static void main(String[] args) {
        ElevatorSystem elevator = new ElevatorSystem(1000.0, 5.0);

        // testcase-VT:
        //     elevator.enterElevator(200.0);

```

```
//      elevator.enterElevator(500.0);
//      elevator.enterElevator(400.0);
//      elevator.calculateEnergyUsage(10.0);
//      elevator.calculateEnergyUsage(25.0);
//      elevator.calculateEnergyUsage(1.0);
//      elevator.printLog();

//      // testcase-FT:
//      elevator.enterElevator(-4.9E-324);
//      elevator.enterElevator(0.0);
//      elevator.enterElevator(1.7976931348623157E308);
//      elevator.enterElevator(-1.7976931348623157E308);
//      elevator.enterElevator(7.754754699406067E306);
//      elevator.enterElevator(9.799190029249484E307);
//      elevator.printLog();

//      // testcase-Z3:
//      elevator.enterElevator(2);
//      elevator.enterElevator(5);
//      elevator.enterElevator(4);
//      elevator.enterElevator(1);
//      elevator.enterElevator(2);
//      elevator.enterElevator(1);

//      // testcase-UVT:
//      elevator.enterElevator(200.0);
//      elevator.enterElevator(500.0);
//      elevator.enterElevator(400.0);
//      elevator.enterElevator(200.0);
//      elevator.enterElevator(500.0);
//      elevator.enterElevator(400.0);
//  }
}
```

● Coverage report

1. Test case number: 6
2. V-Method testing branch coverage: 100%

Element	Coverage	Covered Branches	Missed Branches	Total Branches
▼ VibrationTestingTool	0.7 %	6	838	844
▼ src	0.7 %	6	838	844
▼ code	0.7 %	6	838	844
▼ c35_ElevatorOptimization.java	100.0 %	6	0	6
> c35_ElevatorOptimization		0	0	0
> ElevatorSystem	100.0 %	6	0	6

3. Vibration testing instruction coverage: 97.9%

Element	Coverage	Covered Instructions	Missed Instructions	Total Instructions
▼ VibrationTestingTool	0.7 %	140	20,788	20,928
▼ src	0.7 %	140	20,788	20,928
▼ code	0.7 %	140	20,788	20,928
▼ c35_ElevatorOptimization.java	97.9 %	140	3	143
> c35_ElevatorOptimization	90.0 %	27	3	30
> ElevatorSystem	100.0 %	113	0	113

4. Fuzz testing branch coverage: 66.7%

Element	Coverage	Covered Branches	Missed Branches	Total Branches
▼ VibrationTestingTool	0.5 %	4	838	842
▼ src	0.5 %	4	838	842
▼ code	0.5 %	4	838	842
▼ c35_ElevatorOptimization.java	66.7 %	4	2	6
> ElevatorSystem	66.7 %	4	2	6
> c35_ElevatorOptimization		0	0	0

5. Fuzz testing instruction coverage: 68.5%

Element	Coverage	Covered Instructions	Missed Instructions	Total Instructions
▼ VibrationTestingTool	0.5 %	98	20,830	20,928
▼ src	0.5 %	98	20,830	20,928
▼ code	0.5 %	98	20,830	20,928
▼ c35_ElevatorOptimization.java	68.5 %	98	45	143
> c35_ElevatorOptimization	90.0 %	27	3	30
> ElevatorSystem	62.8 %	71	42	113

6. Z3 branch coverage: 16.7%

Element	Coverage	Covered Branches	Missed Branches	Total Branches
▼ VibrationTestingTool	0.1 %	1	839	840
▼ src	0.1 %	1	839	840
▼ code	0.1 %	1	839	840
▼ c35_ElevatorOptimization.java	16.7 %	1	5	6
> ElevatorSystem	16.7 %	1	5	6
> c35_ElevatorOptimization		0	0	0

7. Z3 instruction coverage: 52.5%

Element	Coverage	Covered Instructions	Missed Instructions	Total Instructions
▼ VibrationTestingTool	0.3 %	74	22,314	22,388
▼ src	0.3 %	74	22,314	22,388
▼ code	0.3 %	74	22,314	22,388
▼ c35_ElevatorOptimization.java	52.5 %	74	67	141
> ElevatorSystem	43.4 %	49	64	113
> c35_ElevatorOptimization	89.3 %	25	3	28

8. Unimproved V-method branch coverage: 33.3%

Element	Coverage	Covered Branches	Missed Branches	Total Branches
▼ VibrationTestingTool	0.2 %	2	838	840
▼ src	0.2 %	2	838	840
▼ code	0.2 %	2	838	840
▼ c35_ElevatorOptimization.java	33.3 %	2	4	6
> ElevatorSystem	33.3 %	2	4	6
> c35_ElevatorOptimization		0	0	0

9. Unimproved V-method instruction coverage: 54.6%

Element	Coverage	Covered Instructions	Missed Instructions	Total Instructions
▼ VibrationTestingTool	0.3 %	77	22,311	22,388
▼ src	0.3 %	77	22,311	22,388
▼ code	0.3 %	77	22,311	22,388
▼ c35_ElevatorOptimization.java	54.6 %	77	64	141
> ElevatorSystem	46.0 %	52	61	113
> c35_ElevatorOptimization	89.3 %	25	3	28